Krishank Kureti
PES2UG23CS285
University Lab DB Lab6

Q.Create a trigger that automatically decreases the total_quantity of an item in the stall_items
table whenever a new row is inserted into the purchased table. (hint: Use AFTER INSERT on
purchased, and update the corresponding record in stall_items.)

Before

SRN	stall_id	item_name	timestamp	quantity
P1010	S4	Chicken Noodle Soup	2022-04-17 13:05:00	3
P1010	S5	Mushroom Risotto	2022-04-17 17:15:00	1
P1010	S6	Shrimp Scampi	2022-04-17 17:20:00	2
P1010	S9	Mutton Stroganoff	2023-04-16 13:10:00	1
P1010	S9	Spinach and Feta O…	2023-04-16 13:10:00	1
P1017	S1	Classic Caesar Salad	2023-04-16 10:05:00	3
P1017	S1	Spinach and Feta O…	2023-04-16 10:05:00	2
P1017	S2	Fish Tacos	2023-04-16 16:00:00	5
P1017	S3	Chicken Noodle Soup	2023-04-16 12:25:00	3
P1017	S3	Vegetable Stir-Fry	2023-04-16 12:25:00	4
P1017	S6	Fish Tacos	2023-07-10 14:00:00	3
P1017	S9	Bacon-Wrapped Shr…	2023-04-16 17:30:00	2
P1024	S4	Chicken Noodle Soup	2022-04-17 13:15:00	1
P1024	S5	Vegetable Pad Thai	2022-04-17 15:00:00	2
P1024	S6	Grilled Steak	2022-04-17 11:05:00	2
P1024	S6	Shrimp Scampi	2022-04-17 11:05:00	3
NULL	NULL	NULL	NULL	NULL

stall_id	item_name	price_per_u...	total_quant...
S1	Caprese Salad	249.00	35
S1	Classic Caesar Salad	349.50	45
S1	Margherita Pizza	350.00	25
S1	Mushroom Risotto	359.00	25
S1	Spinach and Feta Omelette	259.00	40
S1	Vegetable Pad Thai	329.00	25
S1	Vegetable Stir-Fry	299.00	30
S1	Veggie Wrap	399.00	50
S2	Chicken Noodle Soup	529.00	30
S2	Fish Tacos	449.00	25
S2	Mushroom Risotto	387.00	28
S2	Vegetable Stir-Fry	322.00	28
S2	Veggie Wrap	429.00	40
S3	Chicken Noodle Soup	564.50	25
S3	Fish Tacos	481.50	23
S3	Mushroom Risotto	349.00	35
S3	Spinach and Feta Omelette	282.50	38

Code

```
DELIMITER //
CREATE TRIGGER after_purchase_update_quantity
AFTER INSERT ON purchased
FOR EACH ROW
BEGIN
    UPDATE stall_items
    SET
        total_quantity = total_quantity - NEW.quantity
    WHERE
        stall_id = NEW.stall_id AND item_name = NEW.item_name;
END //
DELIMITER ;
INSERT INTO purchased
VALUES ('P1003', 'S1', 'Classic Caesar Salad', '2023-04-16 12:05:00', 5);
```

stall_id	item_name	price_per_u...	total_quant...
S1	Classic Caesar Salad	349.50	40
NULL	NULL	NULL	NULL

SRN	stall_id	item_name	timestamp	quantity
P1002	S1	Classic Caesar Salad	2023-04-16 13:33:00	3
P1003	S1	Classic Caesar Salad	2023-04-16 12:05:00	5
P1017	S1	Classic Caesar Salad	2023-04-16 10:05:00	3
NULL	NULL	NULL	NULL	NULL

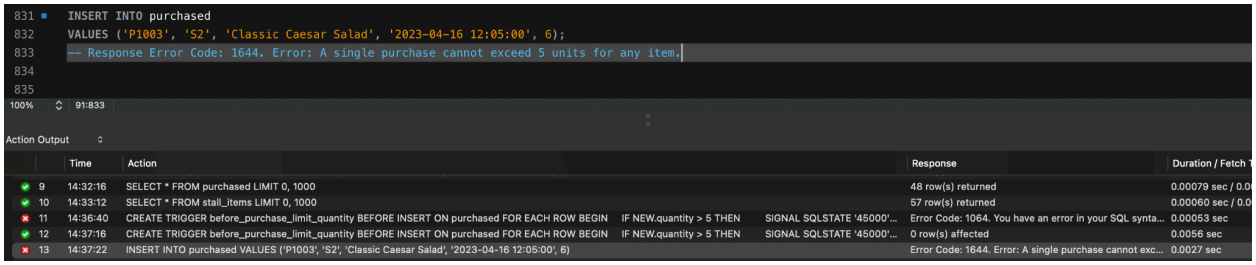
Q.Create a trigger that prevents a participant from purchasing more than 5 units of any single
item in a single transaction (i.e., quantity > 5) in the purchased table.
(hint: Use BEFORE INSERT and raise an error if NEW.quantity > 5.)

Code

```
CREATE TRIGGER before_purchase_limit_quantity
BEFORE INSERT ON purchased
FOR EACH ROW
BEGIN
    IF NEW.quantity > 5 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: A single purchase cannot exceed 5
units for any item.';
    END IF;
END //
DELIMITER ;
```

INSERT INTO purchased
VALUES ('P1003', 'S2', 'Classic Caesar Salad', '2023-04-16 12:05:00', 6);
-- Response Error Code: 1644. Error: A single purchase cannot exceed 5 units for any item.

After



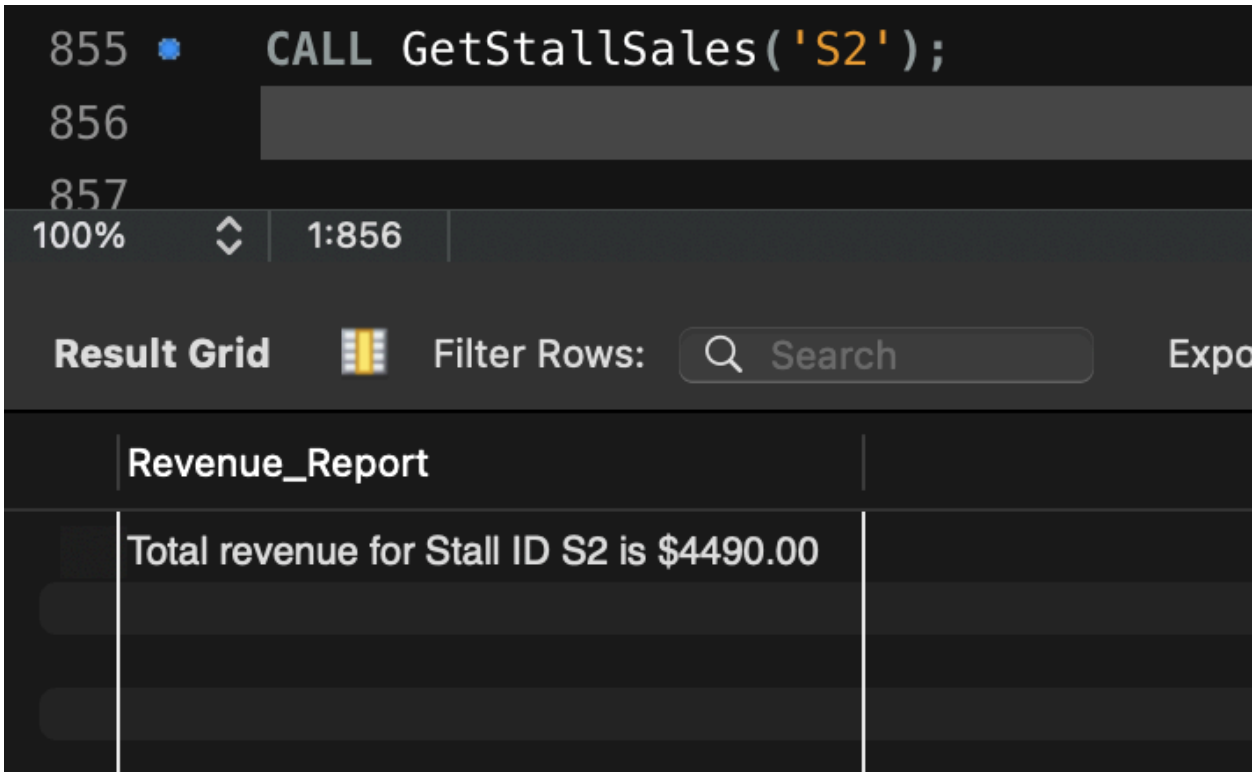
Q.Write a stored procedure GetStallSales that takes a stall_id as input and prints the
total revenue generated from that stall based on the purchased table and item prices
from stall_items.
(hint: Join purchased with stall_items and calculate SUM(price_per_unit * quantity))

Code

```
DELIMITER //
CREATE PROCEDURE GetStallSales(IN stall_id_param VARCHAR(5))
BEGIN
    DECLARE total_revenue DECIMAL(10, 2);

    SELECT SUM(si.price_per_unit * p.quantity)
    INTO total_revenue
    FROM purchased p
    JOIN stall_items si ON p.stall_id = si.stall_id AND p.item_name =
si.item_name
    WHERE p.stall_id = stall_id_param;
```

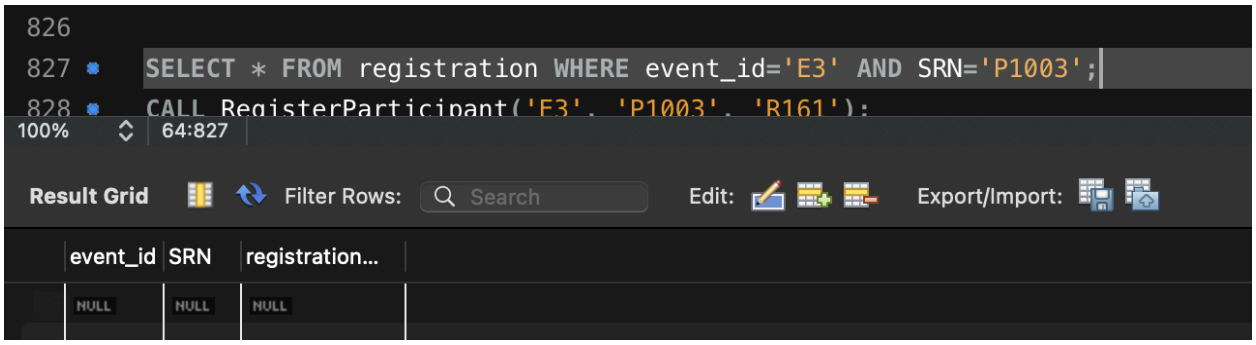
```
    IF total_revenue IS NOT NULL THEN
        SELECT CONCAT('Total revenue for Stall ID ', stall_id_param, ' is $',
total_revenue) AS Revenue_Report;
    ELSE
        SELECT CONCAT('No sales found for Stall ID ', stall_id_param) AS
Revenue_Report;
    END IF;
END //
DELIMITER ;

CALL GetStallSales('S2');
```

After



Q.Create a procedure RegisterParticipant that registers a participant (SRN) for an event
(event_id) by inserting into the registration table. The procedure should take the
event_id, SRN, and registration_id as parameters.
(hint: Use input parameters and basic INSERT.)

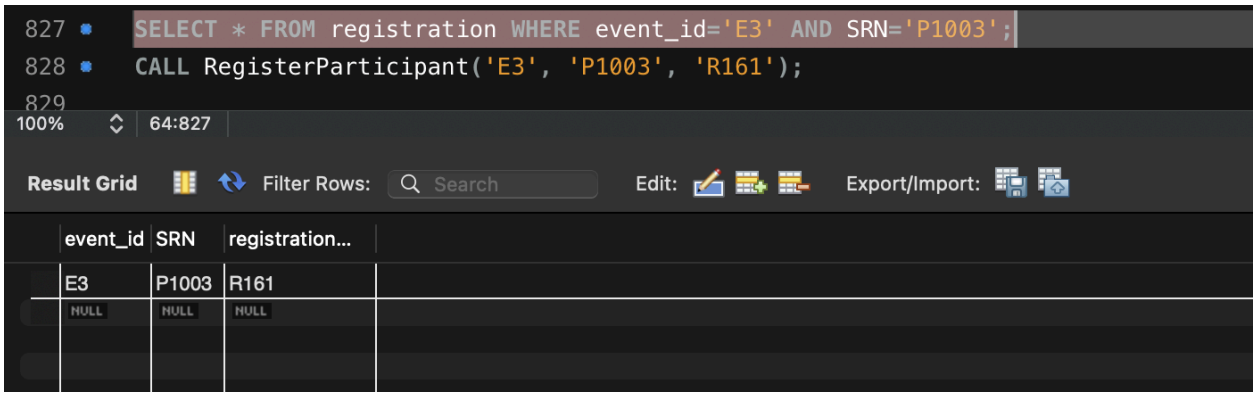
Before



Code

```
DELIMITER //
CREATE PROCEDURE RegisterParticipant(
    IN p_event_id VARCHAR(5),
    IN p_srn VARCHAR(10),
    IN p_registration_id VARCHAR(5)
)
BEGIN
    INSERT INTO registration (event_id, SRN, registration_id)
    VALUES (p_event_id, p_srn, p_registration_id);
END //
DELIMITER ;

SELECT * FROM registration WHERE event_id='E3' AND SRN='P1003';
CALL RegisterParticipant('E3', 'P1003', 'R161');
```

After

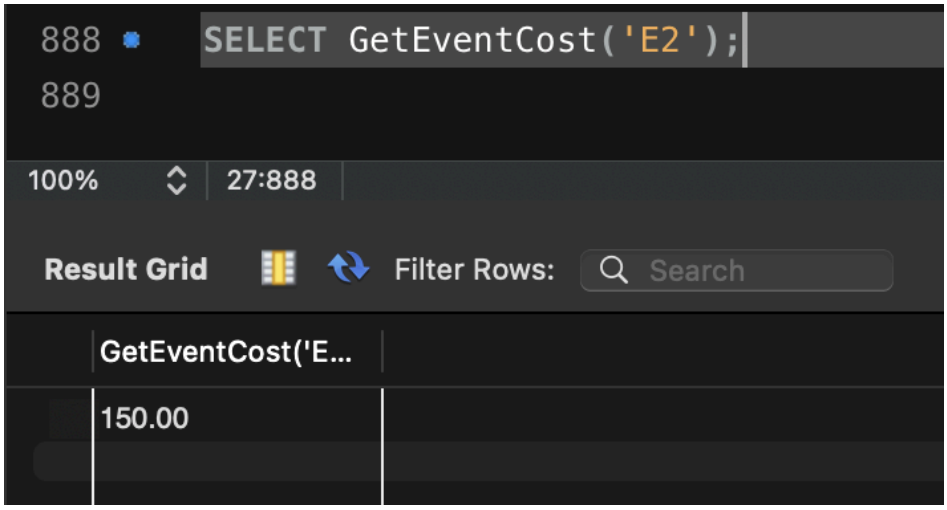


Q. Create a function GetEventCost that accepts an event_id and returns the total price a
participant would pay to register for that event (i.e., return the event's price from the
event table).
(hint: Simple SELECT with RETURN.)

Code

```
DELIMITER //
CREATE FUNCTION GetEventCost(p_event_id VARCHAR(5))
RETURNS DECIMAL(10, 2)
READS SQL DATA
BEGIN
    DECLARE event_price DECIMAL(10, 2);
    SELECT price
    INTO event_price
    FROM event
    WHERE event_id = p_event_id;
    RETURN event_price;
END //
DELIMITER ;
SELECT GetEventCost('E2');
```

After



Q.Create a function GetParticipantPurchaseTotal(SRN) that returns the total amount a participant has spent across all stalls.
(hint: Aggregate using SUM(price_per_unit * quantity) by joining purchased and stall_items.)

Code

```
DELIMITER $$
CREATE FUNCTION GetParticipantPurchaseTotal(p_srn VARCHAR(10))
RETURNS DECIMAL(10, 2)
READS SQL DATA
BEGIN
    DECLARE total_spent DECIMAL(10, 2) DEFAULT 0.00;

    SELECT SUM(si.price_per_unit * p.quantity)
    INTO total_spent
    FROM purchased p
    JOIN stall_items si ON p.stall_id = si.stall_id AND p.item_name =
si.item_name
    WHERE p.SRN = p_srn;
    RETURN total_spent;
END $$
DELIMITER ;

SELECT GetParticipantPurchaseTotal('P1003');
```

After